

CHAP 4: 문자열

문자열 자료형 (`str type`)

문자열 만들기, 특수 문자 (`Escape Characters`),

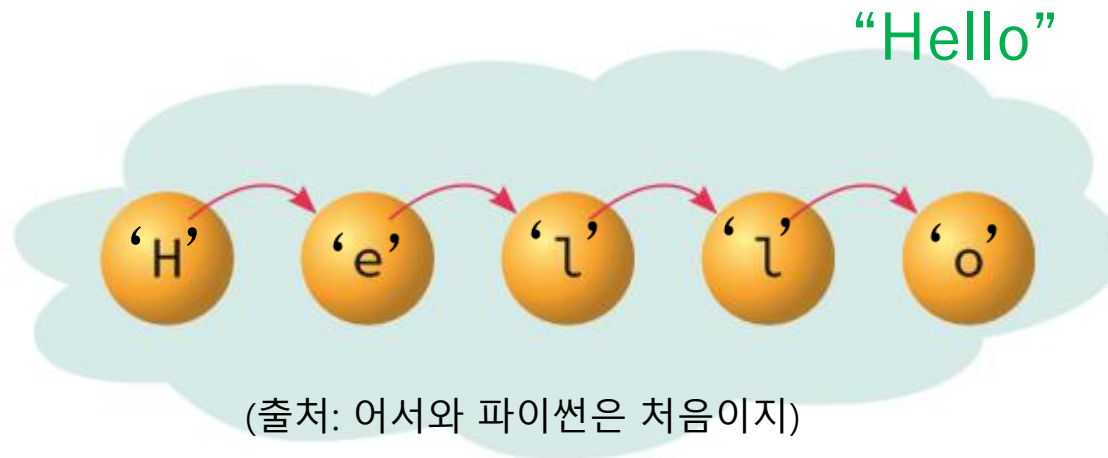
문자열 연산

문자열 인덱싱 (`indexing`), 문자열 `slicing`

문자열 메소드, 문자열 $\leftrightarrow$ 숫자로 변환

문자열 자료형 (str type)

- 문자열(string)은 연속된 문자들의 집합을 의미.
문자들의 나열(sequence of characters)이다.
 - 시퀀스(sequence) 자료형 중 하나이다.
 - indexing, slicing 기법을 사용.



- 파이썬에서는 텍스트를 다루는 자료형으로 **string**을 제공
 - string**은 영어로 끈, 줄을 의미.
문자를 끈으로 가지런히 묶어놓은 것이라고 이해
 - 문자열은 일단 생성하면 그 **내용을 바꿀 수 없다**

문자열 자료형 (str type)

- 문자열을 컴퓨터가 처리하기 위해서 각 문자를 고유한 코드(이진수)로 변환하는데, 이를 문자 인코딩이라고 한다.
- 파이썬은 유니코드를 사용하여 문자열을 인코딩한다.
 - 유니코드(Unicode)는 각 문자를 16-비트 코드(2진수 값)로 표현.
 - 예) 유니코드 표의 일부. 0x0020 ~ 0x006B

2진법	십진법	문자	2진법	십진법	문자	2진법	십진법	문자
010 0000	32	SP	100 0000	64	@	110 0000	96	`
010 0001	33	!	100 0001	65	A	110 0001	97	a
010 0010	34	"	100 0010	66	B	110 0010	98	b
010 0011	35	#	100 0011	67	C	110 0011	99	c
010 0100	36	\$	100 0100	68	D	110 0100	100	d
010 0101	37	%	100 0101	69	E	110 0101	101	e
010 0110	38	&	100 0110	70	F	110 0110	102	f
010 0111	39	'	100 0111	71	G	110 0111	103	g
010 1000	40	(	100 1000	72	H	110 1000	104	h
010 1001	41	)	100 1001	73	I	110 1001	105	i
010 1010	42	*	100 1010	74	J	110 1010	106	j
010 1011	43	+	100 1011	75	K	110 1011	107	k

문자열 자료형 (str type)

- 예) 파이썬의 문자열은 유니코드를 사용하여 인코딩된다.
즉, 0과 1로 이루어진 대응되는 코드 값으로 변환된다.

- msg= 'ABC'



유니코드를 사용하여 인코딩.(2진수 값으로 표현)

msg



0000 0000 0100 0001 0000 0000 0100 0010 0000 0000 0100 0011

유니코드 (2진법)	십진법	16진법	문자
0000 0000 0100 0001	65	0x41	A
0000 0000 0100 0010	66	0x42	B
0000 0000 0100 0011	67	0x43	C

문자열 자료형 (str type)

- 예) 파이썬은 문자열을 유니코드로 인코딩한다.

age = '12'



유니코드를 사용하여 인코딩.(2진수 값으로 표현)

age → 0000 0000 0011 0001 0000 0000 0011 0010

유니코드 (2진법)	십진법	16진법	문자
0000 0000 0011 0000	48	0x30	0
0000 0000 0011 0001	49	0x31	1
0000 0000 0011 0010	50	0x32	2

- 비교(정수)** age= 12

age → 0000 1100 #정수 12 할당. 숫자 12를 2진수로 표현.

문자열 만들기

- 큰따옴표로 양쪽 둘러싸기
- 작은 따옴표로 양쪽 둘러싸기
- 큰따옴표 3개 또는 작은따옴표 3개를 연속으로 양쪽 둘러싸기

- (예) 빈 문자열 만들기
 - 원소가 없는 문자열.
 - 따옴표로만 둘러싸인 문자열

```
>>> msg= ''
>>> msg
''
>>> print(msg)

>>>
```

- (예) 공백 문자열 만들기
 - 공백 문자(스페이스) 1개를 원소로 하는 문자열.
 - 원소 있음!! 공백문자.

```
>>> msg= ' '
>>> msg
' '
>>> print(msg)

>>>
```

문자열 만들기

```
>>> msg1 = 'Hello' Enter
>>> msg1 Enter
'Hello'
>>> msg2 = "Hello" Enter
>>> msg2 Enter
'Hello'
>>> print(msg1) Enter
Hello
>>>
```

```
>>> msg3 = "The true light" Enter
>>> msg3 Enter
'The true light'
>>> print(msg3) Enter
The true light
>>>
```

문자열 만들기

- 큰따옴표(“)로 시작했다가 작은따옴표(‘)로 끝내면 문법적인 오류이다.

```
>>> msg = “Obey to the truth”  
SyntaxError: EOL while scanning string literal
```

영어의 문법 오류처럼
파이썬에도
문법오류가 있어요



문자열 만들기

- 여러 줄의 문자열을 변수에 할당하기 ?

① 줄을 바꿀 위치에 **줄바꿈 문자** ‘\n’ 을 삽입. (\n과 \Wn은 동일.)

진리에 순종하라.
Obedire Veritati.
Obey to the truth.

```
>>> say = "진리에 순종하라.\nObedire Veritati.\nObey to the truth."   
>>> print(say)   
진리에 순종하라.  
Obedire Veritati.  
Obey to the truth.  
>>> say   
'진리에 순종하라.\nObedire Veritati.\nObey to the truth.'  
>>>
```

문자열 만들기

- ② 연속된 작은따옴표 3개 또는 큰따옴표 3개 사용하며, 필요한 위치에서 줄을 바꿔 입력하여 할당한다.

진리에 순종하라.
Obedire Veritati.
Obey to the truth.

```
>>> say = “진리에 순종하라.  
Obedire Veritati.  
Obey to the truth.”  
>>> print(say)  
진리에 순종하라.  
Obedire Veritati.  
Obey to the truth.  
>>> say  
‘진리에 순종하라.\nObedire Veritati.\nObey to the truth.’  
>>>
```

문자열 만들기

- 긴 문자열을 변수에 할당할 때, 여러 줄에 걸쳐 작성할 수 있다.
 - backslash (**₩** 또는 ****로 표시됨) **기호를 사용**하여 작성.
 - 줄 바꿈 없이 연결 됨
 - 예)

예수회 교육은 세계를 긍정한다, 이웃을 위하는
남녀 인간을 양성하고자 노력한다.

```
>>> say = "예수회 교육은 세계를 긍정한다, 이웃을 \
위하는 남녀 인간을 양성하고자 노력한다."
>>> print(say)
예수회 교육은 세계를 긍정한다, 이웃을 위하는 남녀 인간을
양성하고자 노력한다.
>>> say
'예수회 교육은 세계를 긍정한다, 이웃을 위하는 남녀 인간을
양성하고자 노력한다.'
```

특수 문자 (Escape Characters)

- 프로그래밍할 때 사용할 수 있도록 미리 정의해 둔 "문자 조합".
 - 특수 문자 또는 출력 제어 등을 **문자열에 포함**시키기 위하여 제공.
 - **backslash (\)**로 시작하는 두 글자 심볼이다.

backslash notation	Description
\\	백슬래시
\'	작은따옴표
\"	큰따옴표
\n	줄 바꿈 (출력 제어 문자. 출력 때에 적용되어 줄을 바꿈)

- 예)

```
>>> print("우리가 이 보배를\n질그릇에 가졌으니")
우리가 이 보배를
질그릇에 가졌으니
>>>
```

특수 문자 (Escape Characters)

- 예) 문자열에 특수문자가 삽입된 경우

```
>>> print( 'My mother\'s house' ) # 작은 따옴표를 문자열에 포함
My mother's house
>>> print( '\ ' '\ ' '\ ' )
' " \
>>> print( '우리가 \n이 보배를\n질그릇에 가졌으니' )
우리가
이 보배를
질그릇에 가졌으니
>>>
```

- 작은따옴표 / 큰따옴표가 포함된 문자열 만들기 ?
 - 특수문자 사용
 - 문자열에 작은따옴표(') 포함된 경우, 큰따옴표(")로 둘러싼다.
 - 문자열에 큰따옴표(") 포함된 경우, 작은따옴표(')로 둘러싼다.

특수 문자 (Escape Characters)

- 예) 문자열에 작은따옴표(') 포함시키기

the law's requirements

```
>>> say = "the law's requirements" Enter
>>> say Enter
"the law's requirements"
>>> print(say) Enter
the law's requirements
```

- 예) 문자열에 큰따옴표(") 포함시키기

"Come, and you will see." he replied.

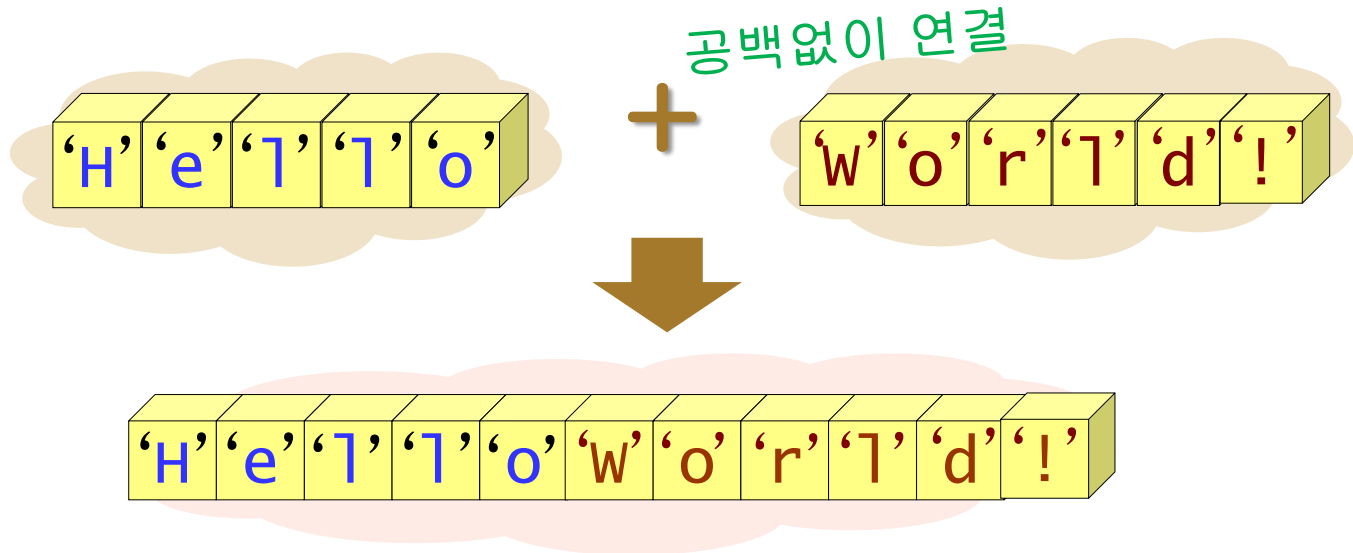
```
>>> say = "Come, and you will see." he replied. Enter
>>>
>>> say Enter
"Come, and you will see." he replied.
>>> print(say) Enter
"Come, and you will see." he replied.
```

문자열 연산

- 문자열에서 **+** 연산자는 연결하기
 - 문자열들을 연결하여 새로운 문자열을 생성.
 - 공백없이 연결 됨.

● (예)

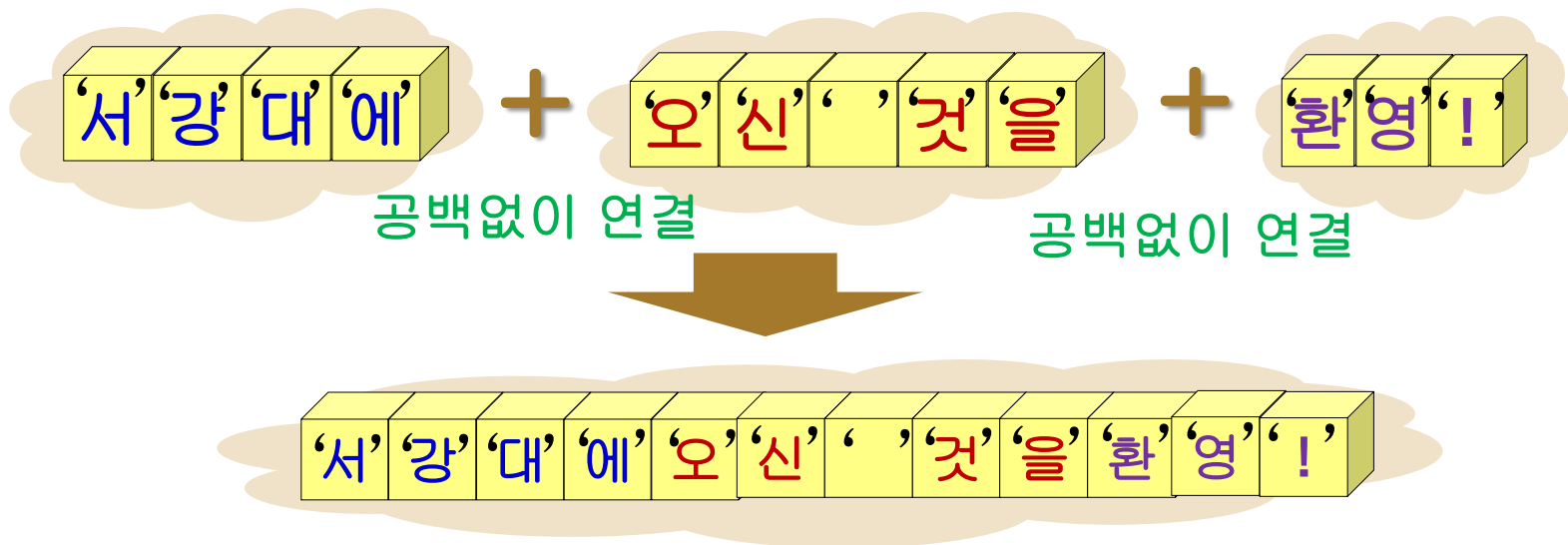
```
>>> 'Hello' + 'World!'   
'HelloWorld!'  
>>>
```



문자열 연산

- (예) 문자열 3개를 연결하기

```
>>> message = " 서강대에" + "오신 것을" + "환영!"  
>>> message  
' 서강대에오신 것을환영!'
```



문자열 연산

- 문자열에서 * 연산자는 반복하기
 - 문자열 * 반복횟수
 - 문자열을 지정된 숫자만큼 반복하여 새로운 문자열을 생성
- 예)

```
>>> message = '축하합니다!'
>>> print( message * 3 ) 
축하합니다!축하합니다!축하합니다!
>>>
>>> print( "=" * 15 ) 
=====
>>>
```

문자열 연산

- **in** 연산자

- 지정된 부분이 문자열 안에 존재하는지를 확인.
- 있으면 **True**를 반환, 없으면 **False**를 반환

- **not in** 연산자

- 지정된 부분이 문자열 안에 존재하지 않는지 확인.
- 없으면 **True**를 반환, 있으면 **False**를 반환

부분 **in** 문자열

부분 **not in** 문자열

```
>>> a = 'Good Morning'
>>> 'Good' in a
True
>>> 'X' in a
False
>>> 'Evening' not in a
True
>>>
```

문자열 연산

- 예) 합격자 명단 확인하기

```
>>> passName= '연주 요셉 길동 철수 영희 소연' Enter
>>> check= '요셉' Enter
>>> check in passName Enter
True
>>> '요셉' in passName Enter
True
>>> '요' in passName #'요'라는 원소가 있어 True
True
>>> '셉' in passName
True          #'셉'이라는 부분값이 있어 True
>>>
```

- 이 경우, 부분을 2문자로 제한(공백문자 포함되지 않음). 그렇지 않으면 정확한 결과를 얻을 수 없다.

😊 좋은 실행 화면

```
Enter: 요셉 Enter
합격!
Enter: 요 Enter
불합격
Enter: 길동 Enter
불합격
```

문자열 인덱싱(Indexing)

- 문자열에서 특정 원소(개별 문자) 추출에 사용
- 문자열은 연속된 문자들로 구성
- 인덱스는 문자열의 각 문자마다 번호를 매기는 것
- 인덱스 범위
 - 양수 인덱스: 0 ~ len(문자열)-1
 - len(문자열) : 지정된 문자열의 길이를 반환

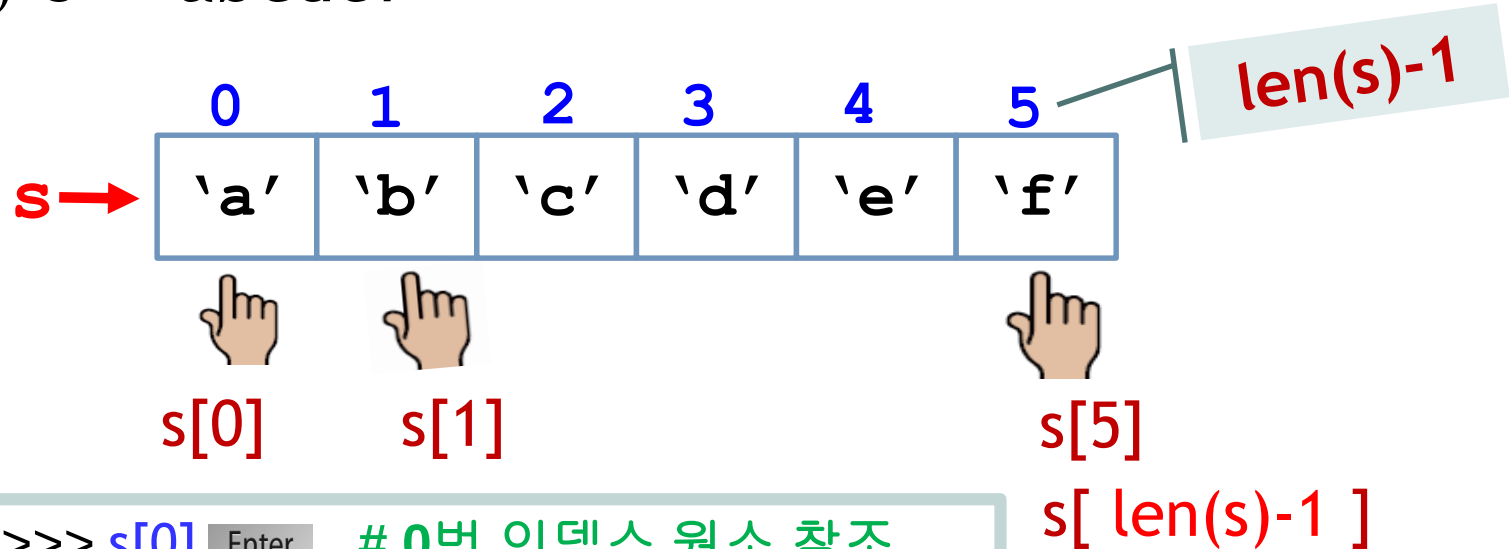
- 예)

```
>>> s = "abcdef" # 문자열 s는 문자 6개로 구성
>>> len(s)       # 문자열의 길이, 원소 개수는 6
6
>>>
```

- 인덱스 번호는 대괄호 [] 안에 기록. 해당 원소(문자)를 참조하게 됨.
 - 예) s[0], s[1], s[2].....

문자열 인덱싱(Indexing)

- 예) `s = "abcdef"`



```
>>> s[0] Enter # 0번 인덱스 원소 참조.  
'a'  
>>> s[1] Enter  
'b'  
>>> s[5] Enter # 마지막 원소. s[len(s)-1]  
'f'  
>>> s[len(s)-1] Enter  
'f'  
>>>
```

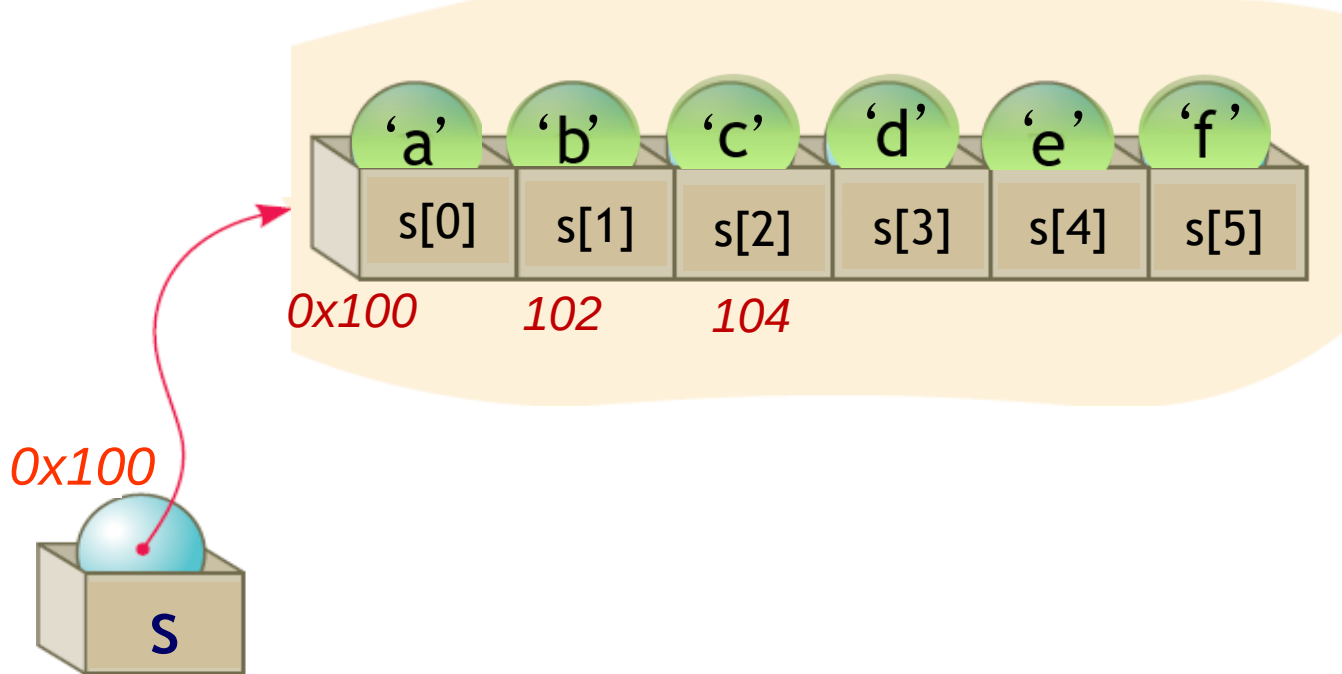
비교 #문자열 전체 참조

```
>>> s Enter  
'abcdef'  
>>>
```

문자열 인덱싱(Indexing)

- `s = "abcdef"`

- 문자열의 인덱스는 0부터.
- `s[0] ~ s[5]`
- 즉, 문자열 내의 원소는
문자열의 첫 원소를 상대적 위치로 해서 참조.

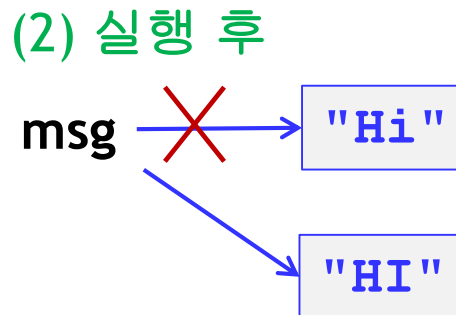
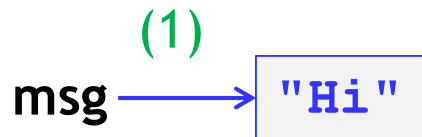


문자열의 참조 값(주소)를 갖고 있음.

문자열 인덱싱 (indexing)

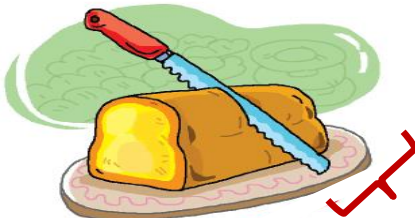
- 문자열은 일단 생성하면 그 내용을 바꿀 수 없다
 - 문자열 객체는 변경불가능. (**immutable** 자료형)

```
>>> msg = "Hi"    #(1) 문자열 할당
>>> print(msg)
Hi
>>> msg[1] = 'I'  # error!! 수정할 수 없다
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    msg[1]='I'
TypeError: 'str' object does not support item assignment
>>> msg = "HI"    #(2) 문자열을 새로 할당
```



문자열 slicing

- 문자열의 인덱스를 기반으로 문자열의 부분 값을 반환
- 문자열에서 일부 문자들 추출에 사용.
- 슬라이싱을 하더라도 원본은 그대로 유지된다.
- 문자열 slicing에는 [] 연산자를 사용.
 - [start : end : step]
 - start: 추출 시작 인덱스. 생략시 **default 값은 첫 원소. 즉, 0**
 - end: 추출의 끝을 지정. (end-1)까지 추출.
생략시 **default 값은 마지막원소까지 추출 즉, len(s).**
 - step: 추출 간격. 생략시 **default 값은 1**



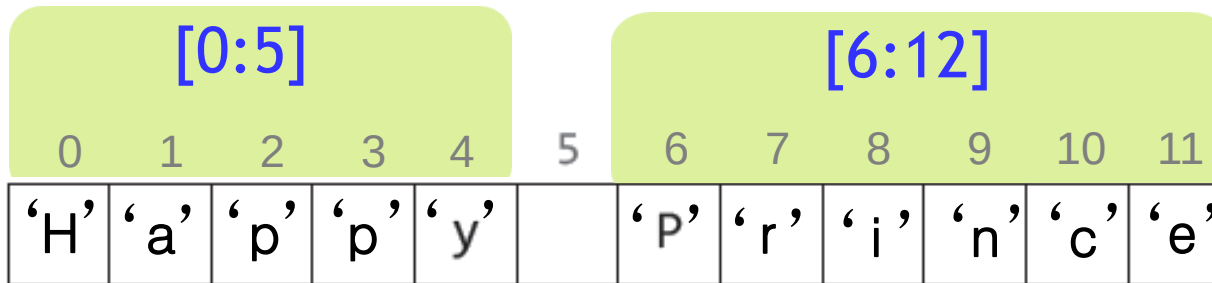
(출처: 두근두근 파이썬)

letters[6:12]

문자열 slicing

- 예) 문자열의 일부를 추출하기(slicing)

```
>>> letters = "Happy Prince"
>>> letters[6:12] # letters[6:12:1], letters[6:] 동일
'Prince'
>>> letters[0:5] # letters[0:5:1], letters[:5], letters[:5:] 동일
'Happy'
>>> a = letters[0:5] # 단어 추출하여 할당
```



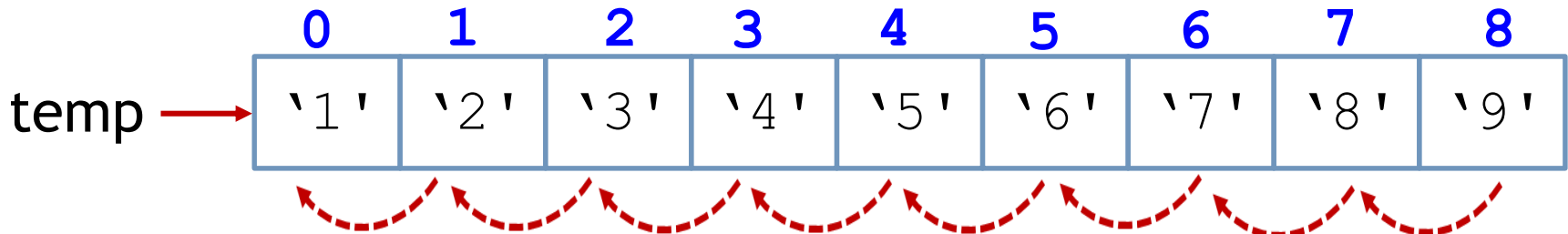
+1
step은 생략시 1.
즉, 추출 간격 +1

문자열 slicing

- 예) step을 -1로 지정하면, 문자열을 거꾸로 추출.
 - step이 음수이면,
start: 생략시 default값은 마지막 원소부터 추출시작.
end: 생략시 default 값은 첫번째 원소까지 추출.

```
>>> temp = "123456789"  
>>> temp[::-1]    # 역으로 슬라이싱 (문자열을 거꾸로)  
'987654321'  
>>>
```

temp[::-1]의 실행



step이 -1이고, -1 -1
마지막원소에서 첫번째 원소까지 추출

문자열 메소드 split()

- split() - space를 기준으로 문자열을 분리.
- 변수이름. split() # 파이썬 코드에서 “.”은 “~의”로 해석
- 공백을 기준으로 분리된 문자열을 반환한다.
- 예) 문자열을 공백으로 분리하고, 각각 변수에 할당

```
>>> a = "Obey to the truth."
```

```
>>> w1, w2, w3, w4 = a.split() # 문자열 a의 split() 함수 실행,  
# 4개로 분리된 문자열을 반환
```

```
>>> w1
```

```
'Obey'
```

```
>>> w2
```

```
'to'
```

```
>>> w3
```

```
'the'
```

```
>>> w4
```

```
'truth.'
```

```
>>>
```

4개의 변수에 할당
w1='Obey', w2='to',
w3='the', w4='truth.'

문자열 메소드 split()

- 문자열 method `str.split()`

```
str.split()  
str.split('구분자')
```

- 구분자를 기준으로 문자열을 분리.
- 구분자를 지정하지 않으면, default로 space를 기준으로 분리.
- 파이썬 코드에서 “.”은 “~의”로 해석
 - `str.split()`는 문자열 객체의 메소드 `split()`
 - 문자열 객체에는 문자열이 제공하는 다양한 메소드들이 포함.
- 예) 구분자 ‘/’를 지정하여 분리

```
>>> newYear = '2019/01/01'  
>>> year, month, day = newYear.split('/')  
>>> year      # year='2019', month='01', day='01'  
'2019'  
>>> month  
'01'
```

문자열 ↔ 숫자로 변환

- 숫자 → 문자열로 변환하려면 `str()` 사용
 - 정수 값을 문자열로 변환
 - 실수 값을 문자열로 변환
- (예) 나이 값으로 출생 년도를 계산하기

```
>>> age=22
>>> print( age , '살')    # 값과 값 사이에 공백이 출력됨
22 살
>>>
>>> print( str(age) + '살') # 값과 값 사이에 공백없이 출력
22살
>>> bornYear = 2022 - age + 1
>>>
>>> print( bornYear , '년생' , age , '살' )
2001년생 22살
>>> print( str(bornYear) + '년생' , str(age) + '살' )
2001년생22살
>>>
```

문자열 ↔ 숫자로 변환

- 문자열 → 숫자로 변환하려면 `int()`, `float()`를 사용
 - `float()` 실수 형태의 문자열을 실수로 변환.
 - `int()` 정수 형태의 문자열을 정수로 변환
- (예)

```
>>> score = '85'    # 정수 형태의 문자열
>>> int(score) + 5   # 정수로 변환 후, 계산에 사용 가능
90
>>>
>>> z = '15.7'
>>> z = float( z )  # 실수 형태의 문자열을 실수로 변환
>>> z
15.7
>>>
```

[참고자료] 문자열 메소드 split()

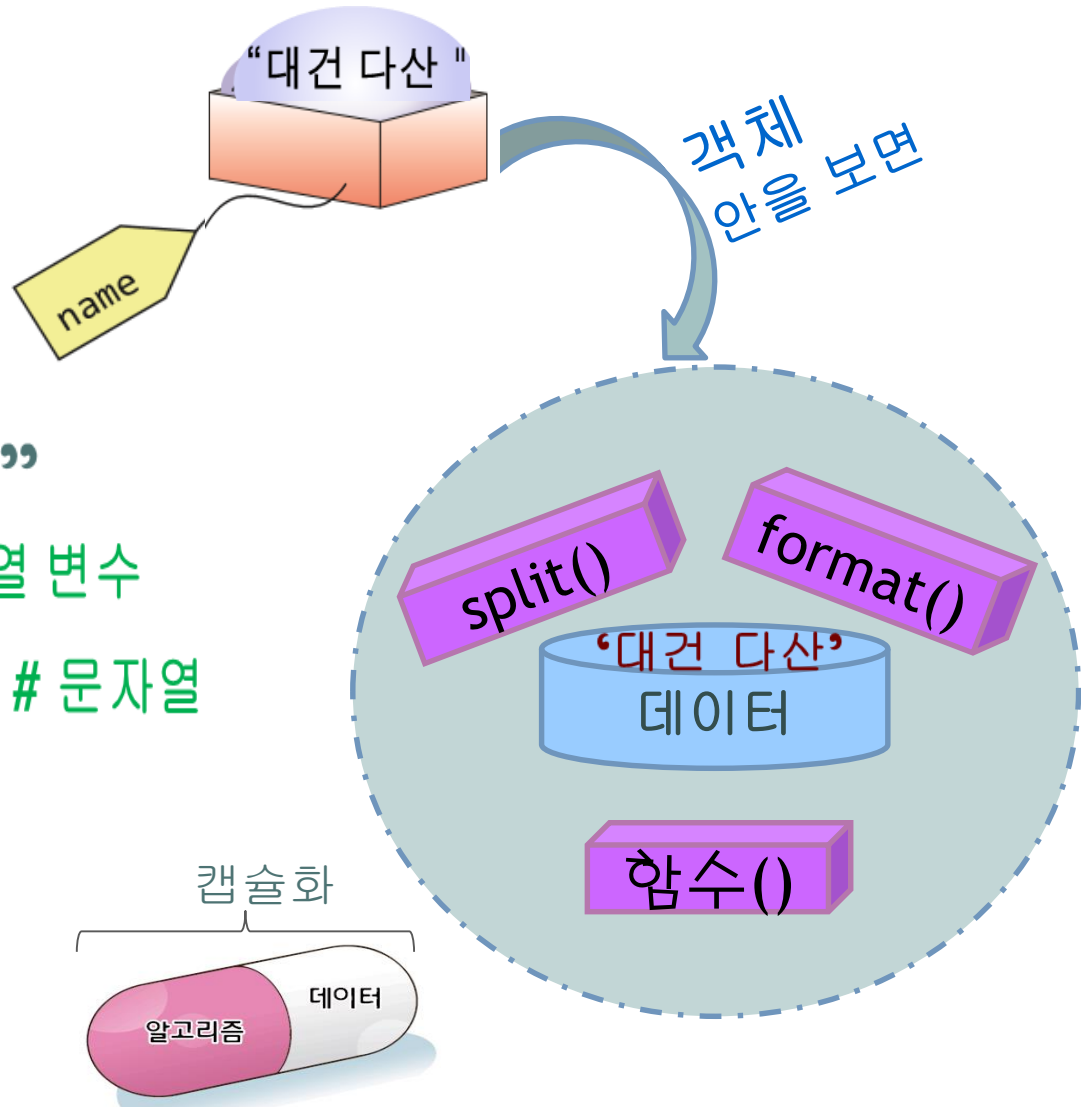
문자열 객체안에는
데이터와 메소드(함수)가
포함되어 있다.

문자열.split()

name = “대건 다산”

name.split() # 문자열 변수

“대건 다산”.split() # 문자열



출처: Power JAVA

[참고자료] 문자열 메소드

메소드	설명
lower()	문자열 데이터를 모두 소문자로 바꾼 문자열을 반환
upper()	문자열 데이터를 모두 대문자로 바꾼 문자열을 반환
islower()	문자열 내의 모든 문자들이 소문자이면, True를 반환
isupper()	문자열 내의 모든 문자들이 대문자이면, True를 반환
isalpha()	문자열이 알파벳(영문, 한글등)으로만 구성되어 있으면 True를 반환
isnumeric()	문자열이 숫자로만 구성되어 있으면 True 반환
isalnum()	문자열이 알파벳과 숫자로만 구성되어 있으면 True 반환
swapcase()	문자열 데이터의 대소문자를 상호 변환한 문자열 반환
title()	각 단어의 제일 앞 글자만 대문자로 변환한 문자열 반환

[참고자료] 문자열 메소드

메소드	설명
replace()	문자열 내에서 지정한 문자열을 새로운 문자열로 바꾼 전체 문자열을 반환
startswith()	문자열이 매개변수로 입력한 문자열로 시작하면 True 반환
endswith()	문자열이 매개변수로 입력한 문자열로 끝나면 True 반환
find()	문자열 내에서 매개변수로 입력한 문자열이 시작하는 인덱스를 반환 (존재하지 않으면 -1을 반환). 여러 번 존재하면 첫 번째 발견하는 인덱스 반환
rfind()	문자열 내에서 매개변수로 입력한 문자열을 뒤에서부터 찾아서 인덱스 반환 (존재하지 않으면 -1을 반환)
count()	문자열 내에서 매개변수로 입력한 문자열이 몇 번 있는지 그 개수를 반환
strip()	문자열 양쪽에 있는 공백을 제거한 문자열 반환 (lstrip() / rstrip(): 왼쪽/오른쪽에서 공백제거)
center(width)	주어진 폭의 가운데 중심으로 정렬된 문자열을 반환 (ljust(width) / rjust(width): 왼쪽/오른쪽 중심으로 정렬)
format(items)	형식을 갖춘 문자열을 만들 때 사용. 문자열을 서식화